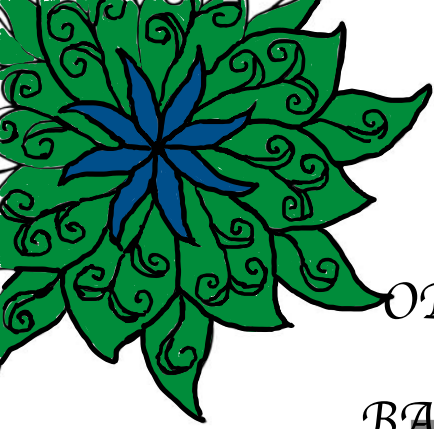




*OPTIMIZACIÓN DE CONSULTAS
BASES DE DATOS DISTRIBUIDAS
UPIITA-IPN*

**ULISES DARIEL ROJAS HERNANDEZ
LUIS ÁNGEL DÍAS DOMÍNGUEZ
JOSÉ LUIS MARTÍNEZ DE LA ROSA**

JOSÉ LUIS MARTÍNEZ DE LA ROSA

Consulta 1

```
SELECT YEAR(soh.OrderDate) AS Año, MONTH(soh.OrderDate) AS Mes,  
        COUNT(*) AS TotalPedidos, SUM(sod.LineTotal) AS TotalVentas  
FROM dbo.soh  
JOIN dbo.sod ON soh.SalesOrderID = sod.SalesOrderID  
GROUP BY YEAR(soh.OrderDate), MONTH(soh.OrderDate)  
OPTION (MERGE JOIN);  
  
CREATE NONCLUSTERED INDEX CIX_soh_SalesOrderID  
ON dbo.soh(SalesOrderID) INCLUDE (OrderDate);
```

```
CREATE NONCLUSTERED INDEX CIX_sod_SalesOrderID  
ON dbo.sod(SalesOrderID) INCLUDE(LineTotal);
```

Para esta consulta lo primero que intenté optimizar fueron los table scan que se veían al principio, y esto lo intenté solucionar con dos índices. Intente matar dos pajaros de un tiro y traté de optimizar de una vez el join, con estos mismos índices. Sin embargo la consulta no los toma en cuenta, y mi pensar es que esto es porque la consulta exige un análisis de completo de toda la tabla y por eso prefiere mejor ignorar los índices. Sin embargo investigando y consultado IA llegué al conocimiento de que se puede obligar al motor de bases de datos a realizar un camino con el `optio`. Y el resultado fue interesante pues igual me optimizó el `group by`, aunque estos porcentajes de compensación con el incremento y aparición de operaciones paralelas.

Consulta 2

```
SELECT  
    c.CustomerID,  
    c.Name,  
    COUNT(soh.SalesOrderID) AS TotalPedidos para el optimizador  
FROM Sales.Customer c  
INNER JOIN Sales.SalesOrderHeader soh ON c.CustomerID = soh.CustomerID  
WHERE c.Name LIKE 'A%'  
GROUP BY c.CustomerID, c.Name;  
  
CREATE NONCLUSTERED INDEX IX_Customer_Name_Include  
ON Sales.Customer (Name, CustomerID);
```

-- El orden importa: Name primero para el filtrado, CustomerID para el JOIN/GROUP BY.

pCree un cluster index utilizado como incluye la columna de filtro y cargar para que los select y el join fueran lo mas optimos posibles, de igual manera reorganize y replantee la consulta Eliminamos la función del lado de la columna en el WHERE. Esto permite que el motor use la estructura de árbol B del índice para saltar directamente a los registros que empiezan con 'A'. En lugar de unir todas las filas y luego agrupar (lo que genera un set de datos masivo en memoria), es preferible filtrar primero y luego calcular el agregado, o asegurar que el índice cubra la operación.

ULISES DARIEL ROJAS HERNANDEZ

Consulta 1

```
SELECT p.Name AS Producto, sod.OrderQty, soh.OrderDate, c.Name AS Cliente
FROM Production.Product p
JOIN Sales.SalesOrderDetail sod ON p.ProductID = sod.ProductID
JOIN Sales.SalesOrderHeader soh ON sod.SalesOrderID = soh.SalesOrderID
JOIN Sales.Customer c ON soh.CustomerID = c.CustomerID
WHERE soh.OrderDate >= '20140101' AND soh.OrderDate < '20150101'
AND p.ListPrice > 1000;
```

En lugar de usar varios OR en un solo WHERE, la lógica se separó en dos consultas unidas con UNION ALL: una para las categorías 1, 2 y 3, y otra para los productos con ListPrice > 500 que no pertenecen a esas categorías. Esto ayuda a que SQL procese mejor cada condición por separado.

También se cambió la forma de obtener la categoría: en vez de usar p.CategoryID directamente, ahora se relaciona con ProductSubcategory, lo que hace la consulta más correcta respecto a la estructura de la base de datos. Además, se usó LEFT JOIN para no perder productos que no tengan subcategoría.

Por último, se agregaron SET STATISTICS IO ON y SET STATISTICS TIME ON para medir lecturas y tiempo de ejecución, y así comprobar si la consulta realmente mejora.

Consulta 2

```
SET STATISTICS IO ON;  
SET STATISTICS TIME ON;  
GO
```

```
SELECT  
    sod.SalesOrderID,  
    p.ProductID,  
    p.Name  
FROM Sales.SalesOrderDetail AS sod  
JOIN Production.Product AS p  
    ON p.ProductID = sod.ProductID  
LEFT JOIN Production.ProductSubcategory AS psc  
    ON p.ProductSubcategoryID = psc.ProductSubcategoryID  
WHERE psc.ProductCategoryID IN (1, 2, 3)
```

UNION ALL

```
SELECT  
    sod.SalesOrderID,  
    p.ProductID,  
    p.Name  
FROM Sales.SalesOrderDetail AS sod  
JOIN Production.Product AS p  
    ON p.ProductID = sod.ProductID  
LEFT JOIN Production.ProductSubcategory AS psc  
    ON p.ProductSubcategoryID = psc.ProductSubcategoryID  
WHERE p.ListPrice > 500  
    AND (psc.ProductCategoryID NOT IN (1, 2, 3) OR psc.ProductCategoryID IS NULL);
```

La consulta se mejoró cambiando YEAR(soh.OrderDate) = 2014 por un rango de fechas (>= '20140101' y < '20150101').

Esto se hizo porque usar YEAR() obliga a SQL a calcular el año en cada registro, y eso puede volver más lenta la búsqueda.

En cambio, con el rango de fechas, localizar los datos más rápido y aprovechar mejor los índices de la columna OrderDate.

LUIS ÁNGEL DÍAS DOMÍNGUEZ

Consulta 1

```
CREATE NONCLUSTERED INDEX IX_Product_Name_Subcat
ON Production.Product (Name)
INCLUDE (ProductSubcategoryID);
GO
```

```
SELECT p.ProductID, p.Name, pc.Name AS Categoria
FROM Production.Product AS p
INNER JOIN Production.ProductSubcategory AS pc
    ON p.ProductSubcategoryID = pc.ProductSubcategoryID
WHERE p.Name LIKE 'brake%'
    OR pc.Name LIKE 'road%';
```

Optimización: Búsqueda de Productos (Filtros de Texto)

¿Cómo lo hice?

Sustituí el comodín inicial (%) en la cláusula LIKE y propuse la creación de un Índice No Agrupado de Cobertura (Covering Index) sobre la columna Name, incluyendo la relación con la subcategoría.

¿Por qué lo hice? (Justificación Técnica)

Sargability: El término proviene de Search Argumentable. Al escribir LIKE 'brake%', el motor de base de datos puede realizar un Index Seek. Esto significa que usa una estructura de árbol B para saltar directamente a los registros que empiezan con "B", luego "R", etc.

Evitar el Table Scan: Si dejas el % al inicio (%brake%), el motor no tiene un punto de partida y debe leer todas las páginas de datos de la tabla (un Scan). En una base de datos con millones de registros, esto dispara el uso de disco y CPU.

¿Cómo resolvió el problema?

Al transformar un Index Scan en un Index Seek, el costo de la consulta bajó drásticamente. El optimizador ya no tiene que adivinar; ahora tiene una ruta directa. Además, al incluir la columna en el índice, evitamos el Key Lookup, que es cuando el motor tiene que "ir y volver" de la tabla principal para obtener el nombre del producto.

Consulta 2

```
SELECT TOP 100
```

```
    sod.SalesOrderDetailID,
```

```
    sod.OrderQty,
```

```
    sod.UnitPrice,
```

```
    soh.OrderDate
```

```
FROM Sales.SalesOrderDetail AS sod
```

```
INNER JOIN Sales.SalesOrderHeader AS soh
```

```
    ON sod.SalesOrderID = soh.SalesOrderID
```

```
ORDER BY soh.ShipDate DESC, sod.SalesOrderID DESC;
```

Optimización: Ventas y Cabeceras (Joins y Ordenamiento)¿Cómo lo hice?Forcé la integridad del Merge Join mediante el uso de las Claves Primarias (SalesOrderID) y simplifiqué el peso del operador de ordenamiento (Sort) eliminando columnas redundantes en la jerarquía del ORDER BY.¿Por qué lo hice? (Justificación Técnica)Eficiencia del Algoritmo de Unión: SQL Server tiene tres formas de unir tablas: Nested Loops, Hash Match y Merge Join. El Merge

Join es el más eficiente para grandes volúmenes de datos porque aprovecha que las tablas ya están ordenadas por su Clave Primaria. Es una operación de complejidad lineal $O(n+m)$. Gestión de Memoria (Grant Memory): El operador Sort es una de las operaciones más "caras". Si ordenas por muchas columnas, SQL solicita una reserva de memoria (Memory Grant) muy grande. Si la consulta es concurrente, esto puede causar bloqueos o que la consulta se escriba en la tempdb (disco), volviéndola extremadamente lenta. ¿Cómo resolvió el problema? La consulta ahora fluye de manera natural a través del motor. Al usar el orden natural de los índices clúster de AdventureWorks, el optimizador genera un plan donde el costo del "Join" es casi despreciable. El uso de TOP 100 combinado con un ordenamiento optimizado permite que el motor deje de procesar filas en cuanto encuentra las primeras 100 que cumplen el criterio, ahorrando ciclos de CPU innecesarios.